

PROJECT REPORT 2

Results and Interpretation

Event-Driven CSV Data Ingestion and Graph-Based Chatbot using FastAPI, Apache Kafka, and Neo4j

Project Title

1. Introduction

This report presents the implementation results and interpretation of the Event-Driven CSV Data Ingestion and Graph-Based Chatbot system. The project was designed to automate CSV data ingestion, enable scalable message streaming using Apache Kafka, store structured information in Neo4j, and provide natural language access to the uploaded data through a chatbot. Unlike traditional database systems that require manual querying, the implemented solution allows users to upload a CSV file and interact with the data using conversational questions.

2. Implementation Summary

The project was successfully implemented as a modular event-driven system consisting of the following components:

- Frontend for CSV upload and chatbot interaction.
- FastAPI backend for API management.
- Pandas for CSV reading and preprocessing.
- Apache Kafka for asynchronous message streaming.
- Kafka Consumer (Loader) for processing incoming messages.
- Neo4j Graph Database for storing structured graph data.
- Cypher Query Engine for retrieving information.
- Docker Compose for deployment and service orchestration.

The implementation ensures that each component is independent, scalable, and easy to maintain.

3. Overall Workflow

The complete workflow of the system is illustrated below.

User

?

?

Upload CSV

?

?
FastAPI Backend
?
?
CSV Validation
?
?
Dataset Cleaning
?
?
Kafka Producer
?
?
Kafka Topic
?
?
Kafka Consumer (Loader)
?
?
Neo4j Graph Database
?
?
User Question
?
?
Cypher Query
?
?
Neo4j
?
?
Answer Returned

4. Results of CSV Processing

The uploaded CRM dataset was successfully processed by the backend.

Validation included:

CSV format verification

Column validation

Duplicate detection

Missing value detection

Empty row removal

Header verification

Cleaning operations included:

- Removing duplicate records
- Trimming unnecessary whitespace
- Standardizing text values
- Filling missing values where applicable
- Formatting dates
- Validating email and phone number formats

The cleaned dataset was saved for further processing.

5. Kafka Results

Apache Kafka successfully functioned as the messaging layer between the backend and the graph database.

Producer Results

- Every CSV row was converted into an individual JSON message.
- Messages were published to the Kafka topic without blocking the application.
- Message serialization ensured structured communication.

Consumer Results

- The loader continuously monitored the Kafka topic.
- Messages were consumed in sequence.
- Every record was transferred to Neo4j for storage.

Interpretation

Using Kafka improved scalability and reliability by decoupling the upload process from database insertion. If Neo4j becomes temporarily unavailable, Kafka retains the messages until the consumer resumes processing.

6. Neo4j Results

The processed data was stored as graph nodes and relationships.

Example entities:

- Customer
- Company
- Product
- Order
- Support Ticket
- Sales Representative

Relationships included:

CUSTOMER_PLACED_ORDER
ORDER_CONTAINS_PRODUCT
CUSTOMER_HAS_TICKET
CUSTOMER_BELONGS_TO_COMPANY
Interpretation

The graph model enabled efficient traversal of connected information and simplified relationship-based queries.

7. Chatbot Results

The chatbot successfully responded to natural language questions by translating user intent into Cypher queries.

Example Questions:

How many customers are from Chennai?
Show all pending support tickets.
Which sales representative handled Customer A?
List all customers belonging to Company X.
Show orders placed during July.

The chatbot returned only information present in the uploaded dataset.

If information was unavailable, the system responded appropriately instead of generating incorrect answers.

8. API Results

POST /ingest

Purpose:

Uploads the CSV file and begins the ingestion process.

Output:

Upload status
Validation result
Number of processed rows

POST /chat

Purpose:

Receives a natural language question.

Output:

Generated Cypher query

Database response

Final chatbot answer

GET /status

Purpose:

Displays ingestion progress.

Output:

Total rows processed

Remaining rows

Completion percentage

GET /health

Purpose:

Checks service availability.

Reports:

Backend status

Kafka status

Neo4j status

9. Performance Interpretation

The implemented architecture demonstrated several strengths.

Fast Data Processing

CSV records were streamed efficiently through Kafka without blocking the upload service.

Asynchronous Processing

Database insertion occurred independently of user interaction, reducing waiting time.

Scalability

The producer and consumer architecture supports future expansion without significant redesign.

Reliability

Kafka ensured message persistence until successful consumption.

Data Consistency

Using MERGE prevented duplicate nodes and relationships.

10. Advantages

Automated CSV ingestion.

Dynamic dataset support.

Graph-based storage.

Natural language querying.

Modular architecture.

Easy deployment using Docker.

Scalable event-driven design.

Improved maintainability.

Separation of concerns.

Reliable message delivery.

11. Limitations

The current implementation has several limitations.

Supports CSV files only.

Basic natural language interpretation.

Single Neo4j instance.

No authentication mechanism.

No cloud deployment.

No role-based access control.

Limited analytics dashboard.

These limitations provide opportunities for future enhancements.

12. Future Enhancements

The system can be extended with the following features.

Excel and JSON support.

PDF data extraction.

Authentication and authorization.

Cloud deployment using Kubernetes.

Large Language Model integration.

Advanced semantic search.

Real-time analytics dashboard.

Multi-user support.

Automated schema detection.

Data visualization tools.

13. Conclusion

The implementation successfully fulfilled the objective of building an event-driven CSV data ingestion platform integrated with a graph database and a natural language chatbot. Apache Kafka enabled reliable asynchronous communication, Neo4j provided efficient graph storage, and FastAPI coordinated all system components. The resulting architecture is modular, scalable, and suitable for handling dynamic datasets while allowing users to retrieve information through conversational interactions.